

Software Project Management – IS212

*specifying system **behaviour***

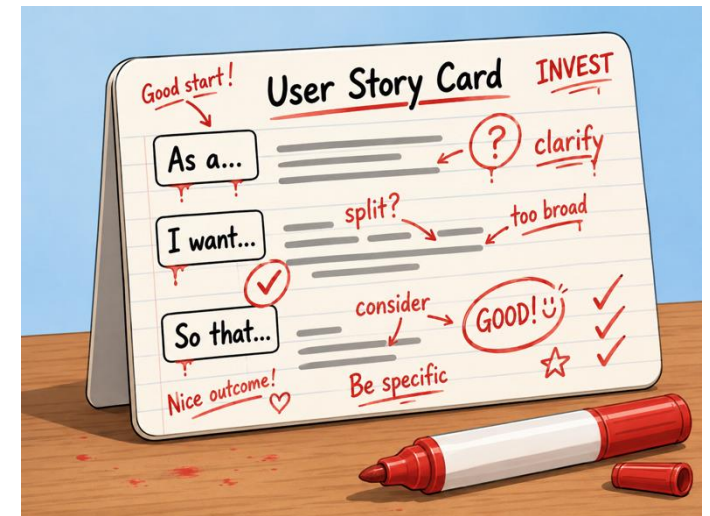
Test Case Design & Software Architecture

*specifying system **structure***

Announcements

Announcements – User Story Assignment Feedback

- We have published the **formative feedback** for your **User Story Assignments**
 - *Thank you for your efforts – great quality stories overall!*
- Please view your (latest) submitted attempt, where it should be visible under the overall “Attempt Feedback”
- Note that our feedback is on the **overall quality of the stories (“INVEST”)** and does **not** take into account any customer clarifications or changes
- **Do approach us if you have any questions** or if anything in the feedback is unclear



Announcements – Project Instructions

- The **Project Instructions** are out – please download from eLearn!
- Start figuring out **your team's process**
 - *Which tools will support you? How will you conduct your meetings?*
 - *How long will your sprints be? How will you document your process?*
- Need any clarifications?
 - *Clarifications on project instructions —> **#help-is212** on Slack*
 - *Clarifications on customer requirements —> via your section's customer briefings only*
- Plan to **start** your first sprint **this week**
- Plan to **end** your final sprint **by Friday Week 12 (at the latest)**
 - *All deliverables are due **Friday Week 12 @ 23:59***

Announcements – Week 5 Quiz

- **Week 3 Quiz results are out** – *please ask us if any doubts*
 - *Of the 4 quizzes, only your top 3 quizzes retained*
- Our second quiz will take place at the **start of your Week 5 class**
 - *Starts promptly at the beginning of class!*
 - *No extra time if you arrive to class late*
- Need to be physically present (no make-ups; see Week 1 slides)
- Requires Respondus LockDown Browser
- Short (~10 mins); covers the **Weeks 3–4 official materials**
 - *MCQ-style questions (no written)*

Recap

Week 3 Recap – Agile Estimation

- What is the difference between an **estimate** and **estimation**?
- Why do agile teams prefer **relative estimation** (e.g. story points)?
- How do we estimate a product backlog, and measure a team's **velocity**? 
- Beyond velocity – what do **flow metrics** (cycle time, throughput) add?
- How does **Monte Carlo forecasting** turn throughput into a probability?
- How do **AI coding assistants** change the estimation picture?

Week 3 Recap – Scrum Events

start of a sprint

Sprint Planning Meeting

during a sprint

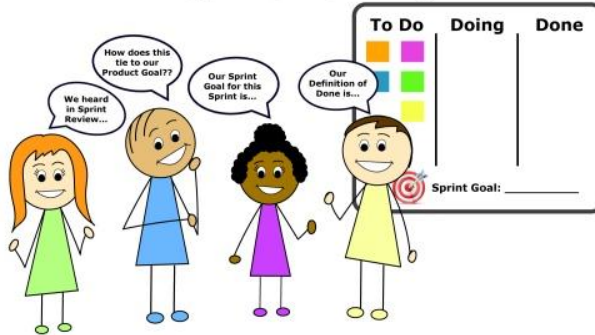
Daily Scrum Meeting

end of a sprint

Sprint Review Meeting

Sprint Retrospective

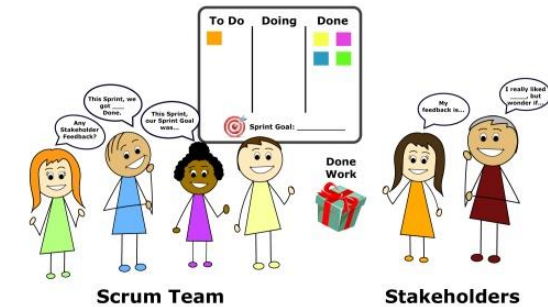
Sprint Planning



Daily Scrum



Sprint Review



Today: Test Case Design & Software Architecture

Objectives

- Explain how testing fits into an agile software development process
- Derive **test cases** from the acceptance criteria of user stories (positive, negative, boundary)
- Explain the rationale for **visualising software architecture**
- Model the context, containers, and components of a system using the **C4 model**
- Use **AI assistants** to accelerate and critique both test design and architecture views

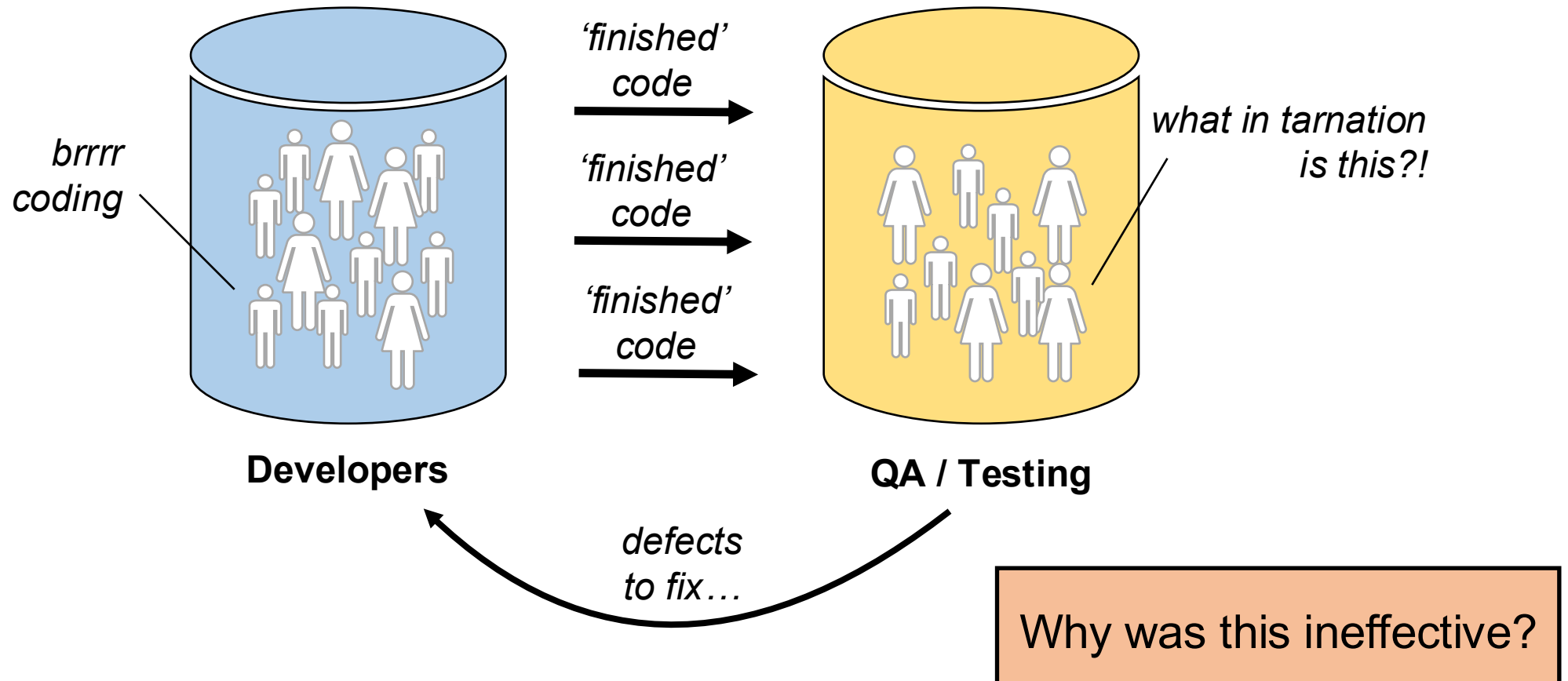
Outline

- **Part 1 – Test Case Design:** traditional vs. agile testing; the 5-step approach; test cases in scrum; AI in test design
- **Part 2 – Software Architecture:** why architecture; the C4 model (C1–C4); AI and architecture

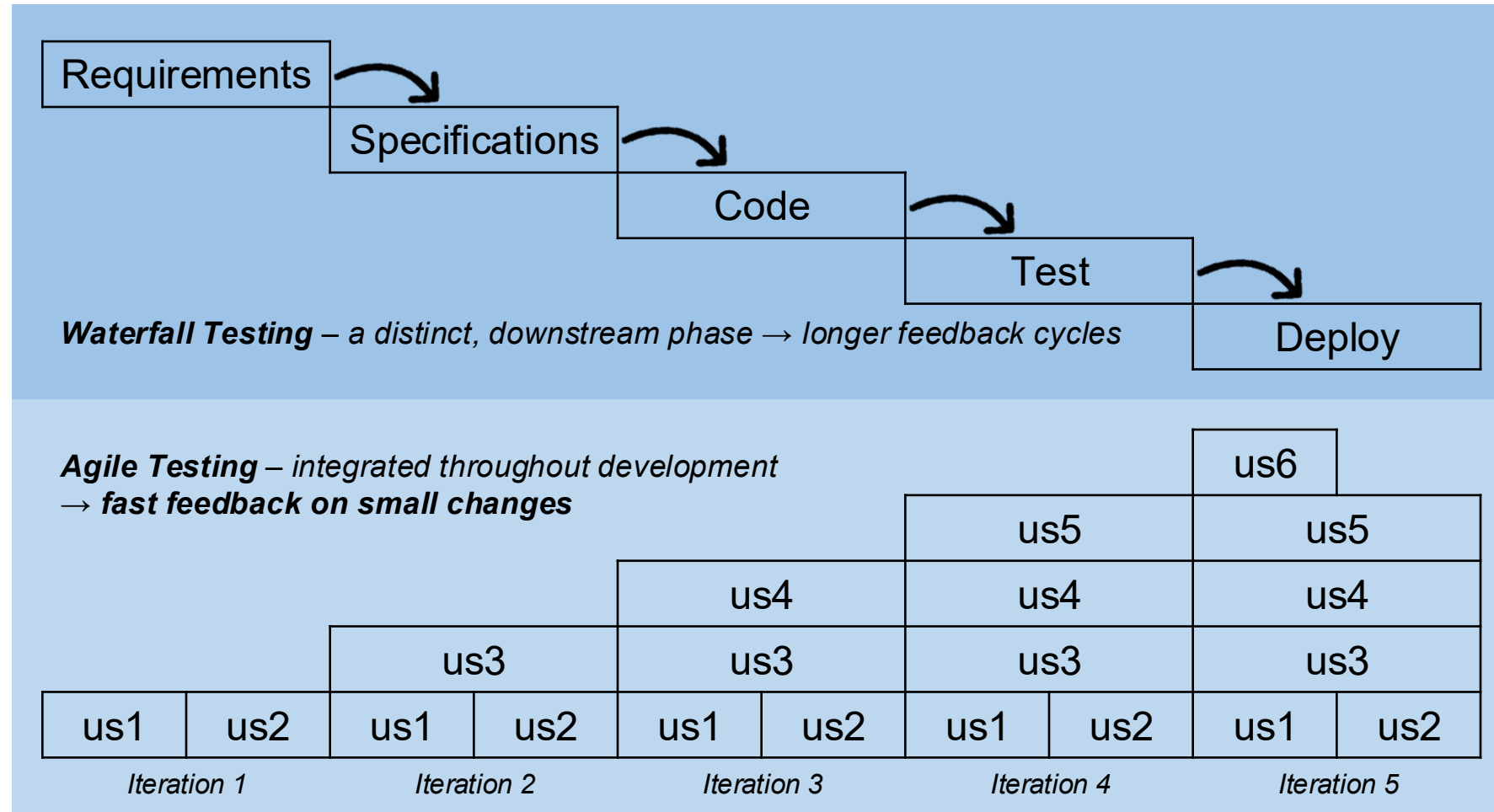
Software Testing

How Not to do Testing

- In the past, developers and testers worked in their **own silos**



What do Agile Practitioners do Differently?



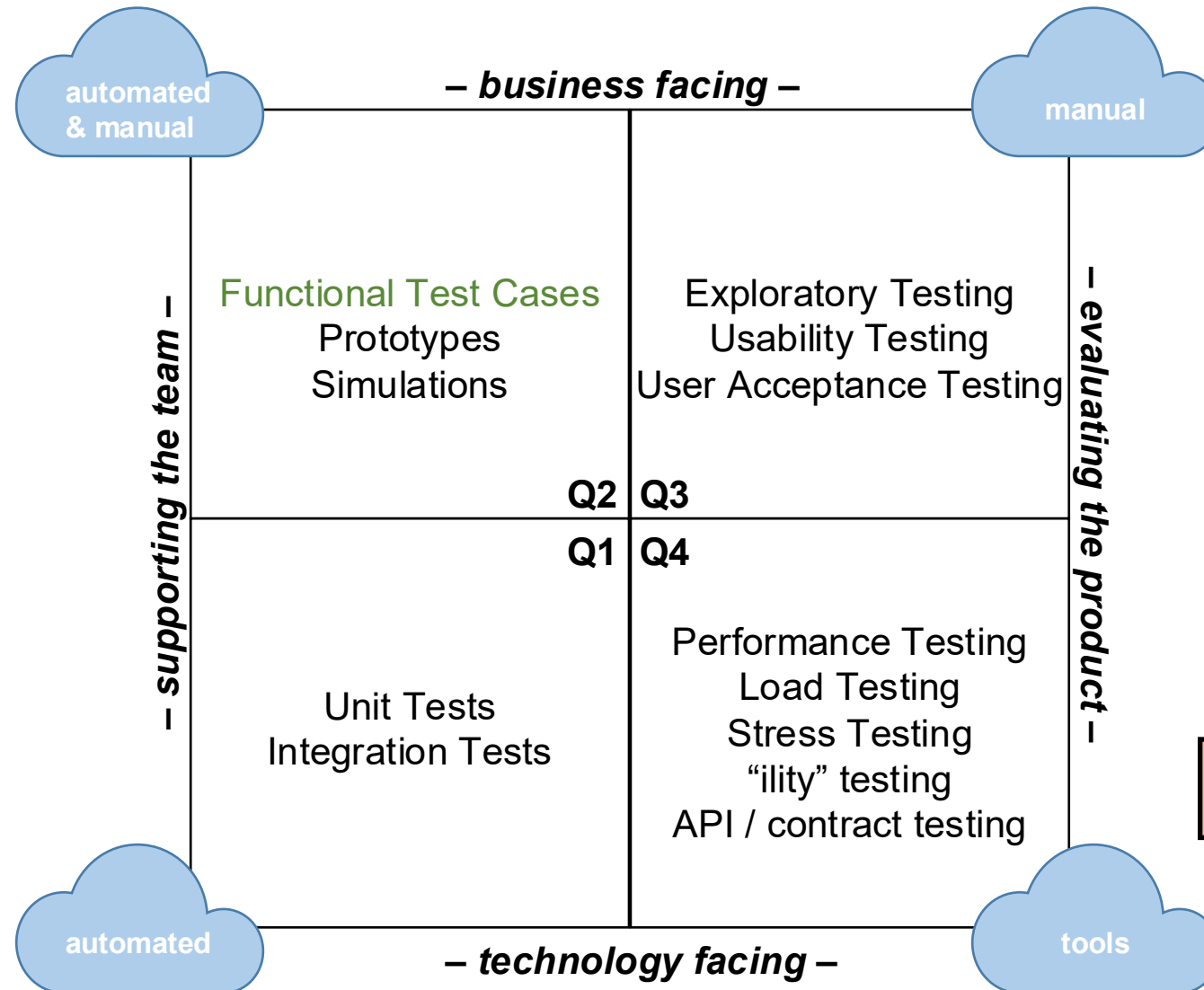
What do Agile Practitioners do Differently?

- Testing is a core activity in **every iteration** (i.e. sprint)
- Testers and developers (possibly the same people) work closely together in **cross-functional teams** (i.e. the scrum team)
 - *Break down silos*
 - *Build a shared understanding*
 - *Frequent and regular feedback (e.g. sprint reviews)*
- Developers **frequently integrate code** into a shared repository
 - *Frequent, smaller changes → diagnose issues earlier & easier fault localisation*
 - *AI accelerates code generation, but **keep integrations small and reviewable***
- **Automation of tests** supports **continuous integration (week 6)** and safe **refactoring (week 11)**



In what ways can a system be tested?

Agile Testing Quadrant



Which tests matter?

Which Tests Matter?

- For this course project, at minimum we need **test cases** for your user stories
 - *i.e. **detailed** description of steps/conditions to verify that a function works as intended*
 - *Shared understanding of the expected behaviour of the feature*
 - *Facilitates **user story acceptance** by the Product Owner (and/or end users)*
 - ***Constrain the agents** in AI-assisted code generation*
- **Automated unit/integration tests** for user stories are also valuable
 - *Helps ensure that new code changes can be integrated ‘safely’*
 - *Facilitates **regression testing** – maintain integrity of the software over time*
- Beyond this... **what does the customer want?**
 - *Is there a performance / scalability / accessibility / ... requirement?*
 - *Is a potential production outage a mild inconvenience, or a serious threat (\$\$\$) to the business?*

Example – Stress Testing in Production

- **Netflix** – introduced tension in production to test the “resilience” of their recovery systems
 - E.g. **Chaos Monkey** – randomly kills processes and compute servers in production
 - (Why production and not a test environment?)
 - “**Chaos engineering**” – a classic approach that helped establish the field
- Testing depth follows business risk – an outage here costs real money (\$\$s)

NETFLIX



Test Cases

Recap – User Story Card

Title <i>Good short title</i>	Priority <i>Order of implementation</i>	Estimate <i>Story point value</i>
User Story <i>Simple description (no details) of customer/user need (conversation starter)</i> <i>As a <role>, I want to <perform an action> so that I can <achieve a goal or outcome></i>		
Acceptance Criteria <i>List of conditions fulfilled on completion of implementation</i>		



- **Boundaries of the story**
- **Use to check if the story ‘done’**

Recap – User Story Card

Title <i>Accept Credit Card Payments</i>	Priority ...	Estimate ...
User Story <i>“As a customer, I want to pay by credit card online, so that I can complete my purchase conveniently.”</i>		
Acceptance Criteria Ensure that the customer: • Can input a valid card number, expiration date, CVV, and cardholder name • Can <u>only</u> pay with Visa, Mastercard, or American Express • Is requested to provide an OTP • Has their payment processed and their order confirmed • Is informed of payment success or failure within 2 seconds • Has a secure TLS connection		

Can we translate these to tests?

Test Cases

- A **test case** is a **specific, detailed** description of the steps and conditions required to determine whether a function or feature **works as intended**
- Broadly, they exist to **confirm that the acceptance criteria (ACs) have been met**
- Unlike ACs, functional test cases consider the **specific outputs** of the application **given specific inputs**
 - Test cases must be **clear and unambiguous** enough that anyone can follow
 - E.g. “**log in to the app**” ❌ vs. “**log in with username X and password Y**” ✅
- A test case describes **inputs, conditions, and expected behaviour**. It may be executed manually **or** encoded as an automated test
 - Today we design functional cases; Week 6 & 9 show how automate execution

Test Case Template

Item	Description	Example
Test Case ID	Unique ID of the test case	TC-001
Test Scenario	Succinct summary/objective of the test case	Check login with valid credentials
Pre-conditions	Conditions to be fulfilled prior to executing the test, e.g., database must contain right test data	Run tc1_setup.sql on testDB
Test Steps	Step-by-step procedure to execute the test; this is what the tester must follow	1. Navigate to https://abc.com/login 2. Enter UserId and Password 3. Click Submit
Test Data	The specific inputs used to conduct the test	UserId=user99; Password=pass99
Expected Result	Expected result from the test	User logs in successfully; home page shown
Created By*	Author of test case	John T.
Date of Creation*	Date test case created	22/02/2024
Item	Description	Example
Actual Result	Actual result from the test (to be filled post-test)	User logs in successfully; home page shown
Pass/Fail/Not Executed/Blocked	Status of the test	Pass
Remarks	Any comments on the test	NA
Executed By*	Person who ran test case	John T.
Date of Execution*	Date test case was run	22/03/2024

Why is it important to reset the test database?

■ **Test case specification** – written once ■ **Test execution record** – one per run * optional metadata

One Specification, Many Executions

- The **test case specification** is written once, and changes only when the *requirement* changes
- Every run adds a **test execution record** – build/environment, actual result, status, evidence or defect link
- The same case is **executed again and again** to detect **regressions** across browsers, builds, environments, sprints
 - *A passing case is only ever “passing as of that build”*
- For your project, record the specification fields plus the **latest execution status and evidence**
- The template is just **one useful representation** – the transferable skill is: **specific inputs + preconditions + action + expected result**

Deriving Test Cases from ACs

- Notice the similarity between **scenario-based ACs** and **test cases**

Acceptance Criterion	Test Case
Given...	Pre-conditions...
When...	Test Steps/Test Data...
Then...	Expected Result...

- While not a compulsory step, it can be a **helpful exercise** to first translate checklist-based ACs into scenario-based ACs in order to derive test cases
 - *Recall that a checklist-based AC can be refined into **multiple** scenario-based ACs...*
 - *...and thus **multiple** different functional test cases*

Example – ACs for Valid Passwords

- To illustrate, consider the many scenarios (and thus test cases) associated with entering a “valid” password

How many test cases do we need?

STRONG PASSWORDS ACCEPTANCE CRITERIA

Data	Expected result	Expected message
Aa9ab\$\$	Fail	Too short
AAbbCC11	Fail	No special characters
\$\$\$bbb111	Fail	No upper case
AAA%%1111	Fail	No lower case
AAA%%111111	Fail	No numbers
AAAA(((bbb	Fail	Braket is not a special character
BBBBB}}}}hhhh	Fail	Brace is not a special character
256 characters input	Fail	Max password length is 255
IsThis\$AGood11	Pass	
IsThis~Good11	Pass	

Identifying Test Cases

1. Visualise Workflow

2. Create "Happy Path"

3. Cross-cutting
Quality Expectations

4. Negative Testing

5. Boundary Testing

User Story

"As a customer, I want to pay by credit card online, so that I can complete my purchase conveniently."

Acceptance Criteria

Ensure that the customer:

- Can input a valid card number, expiration date, CVV, and cardholder name
- Can only pay with Visa, Mastercard, or American Express
- Is requested to provide an OTP
- Has their payment processed and their order confirmed
- Is informed of payment success or failure within 2 seconds
- Has a secure TLS connection

1. Visualise Workflow

- Put yourself in the **end user's shoes** 
- **Visualise what they see** and how they **might behave**, and derive test cases for these different behaviours
- What happens when they...
 - *Click “Order History”?*
 - *Click “Add to Cart”?*
 - *Write their name in the card number field?*
- If you're not familiar with the feature, converse with the team, or navigate through other existing features to get a 'feel'

Can we generate test cases before the feature is built?

2. Create “Happy Path”

- What is the path through a feature that encounters **no errors or exceptions**? (Also known as the “**positive test case**”.)
- For our user story, this would be a test case in which the user **successfully pays** for a (non-empty) cart with a credit card
 1. Select cart item(s)
 2. Click checkout
 3. Select card payment
 4. Card payment screen is shown
 5. Enter card details
 6. Click submit
 7. Purchase confirmation screen is shown
- Test cases derived from the happy path are usually the **first to be executed** when validating a feature (**Why?**)

3. Cross-cutting Quality Expectations

- Are there **cross-cutting expectations** that should hold across *many* features? For example:
 - *authentication & authorisation · security requirements*
 - *accessibility · responsive UI · browser / device compatibility*
 - *consistent error handling · common navigation & design-system behaviour*

*Write the same test for every feature
– or set the bar once in the **DoD**?*

- **Don't** duplicate generic checks for every story – but do create a dedicated test when the cross-cutting concern **produces story-specific behaviour**

4. Negative Testing

- We **can't** expect the happy path to be the only path
- Sometimes, users enter **erroneous data**. Sometimes, business **exceptions occur**. And sometimes, **systems are unavailable**. E.g.:
 - *User enters their name in the credit card field*
 - *User enters an (unsupported) Discover card number*
 - *A successfully ordered item becomes out of stock before dispatch*
 - *The payment vendor (e.g. Stripe) is unavailable*
- We need test cases to verify that these “**sad**” events are handled **correctly**, e.g. through **appropriate error messages**

Classlist

[Add Participants](#) ▾[Enrollment Statistics](#)[Email Classlist](#)[All](#)[Students](#)[Instructors](#)[Teaching Assistants](#)[Auditor](#)

View By:

[Sections](#) ▾

Sections:

[G1](#) ▾[Apply](#)[Show Search Options](#)

No Search Results

[Clear Search](#)

Total Users: 0

Aside: is this a negative test case?



5. Boundary Testing

- Does the scenario involve **ranges of inputs**?
 - Are there **boundary values** at which the **expected behaviour changes**?
E.g., a \$20 minimum order:
 - A \$19.99 order 
 - A \$20.00 order 
 - A \$20.01 order 
- “Just below, exactly at, and just above”*
- Bugs / logical errors are common around boundary values – **why?**
 - This is **boundary-value analysis**; its partner is **equivalence partitioning**
 - *Group inputs that should behave alike, test one from each group – then test the boundaries between them*



By entering the site you agree with our
Privacy Policy and Terms & Conditions.

[Privacy](#)
[T&Cs](#)

Singapore 

DD MM YYYY

UNCAGE



*What's the **boundary**
we need to test?*

Item	Content
Test Case ID	
Test Scenario	
Pre-conditions	
Test Steps	
Test Data	
Expected Result	

AI & Test Case Design: Generate, Then Curate

- AI assistants can draft test cases straight from a user story and its ACs – often dozens in seconds
- Strong on the obvious cases: **happy-path** (②) and **cross-cutting** (③)
- Less reliable where the relevant domain context or business rules are **implicit**: **domain boundaries** (⑤) and **business-rule negatives** (④)
- So the 5-step process becomes a **review skill** – AI generates more tests; you judge whether they are the right ones

*Your job shifts from writing to **interrogating**:
do the tests **actually** cover the story?*

The Confidence Trap

- AI-generated cases read as confident and complete – even when **neither**
- A passing test case only matters if it checks the right behaviour
- Coverage "theatre" – many cases, all exercising the same happy path
- When AI writes **both the code and its tests**, both can share the same blind spot
 - You are *ultimately responsible* for the judgement AI can't make:
what behaviour is required · which risks matter · whether each expected result is actually correct

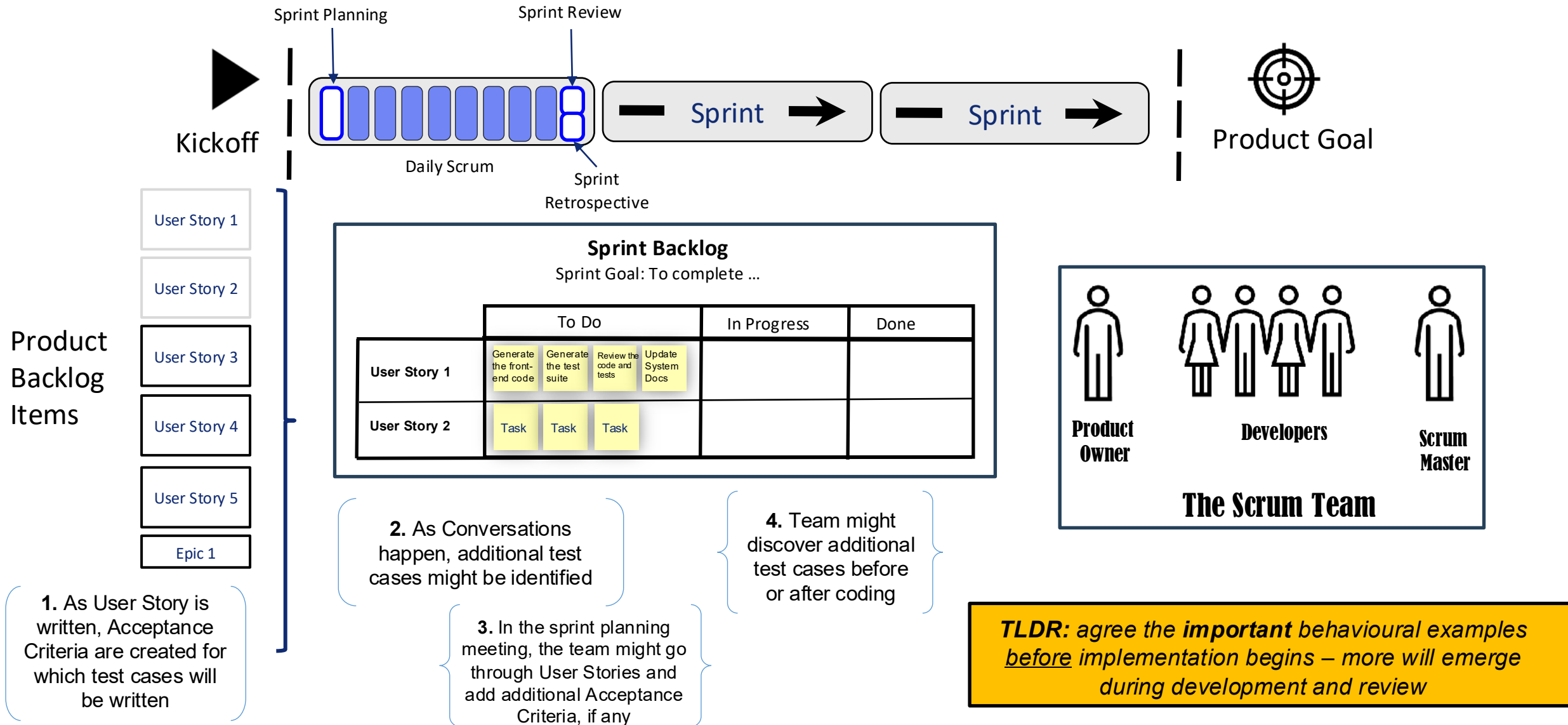


Test Cases as Executable Spec

- Flip the order:
 - *(1) agree the test cases first;*
 - *(2) generate code to pass the tests* 
- The cases become a precise, checkable **specification** the AI agent must satisfy
- This is the idea behind **spec-driven development**
- Well-formed ACs → **concrete test cases** → a tighter brief & **fewer misunderstandings** by the AI

Test Cases in Scrum

When to Write Test Cases?



ACs → Test Cases → Definition of Done

- **Acceptance Criteria** → what must this story do?
- **Test Cases** → what concrete **evidence** will show that behaviour?

Traceability: can we tell which requirement each test case is evidence for?

- **Definition of Done** → what broader quality and cross-cutting concerns must hold before the Increment is “Done”?

Definition of Done (DoD)

- The **Definition of Done (DoD)** is a shared quality standard for the Increment
 - *If the organisation does not already prescribe one, the **Scrum Team** creates one appropriate for the product*
- A DoD is typically a checklist, e.g.:
 - *Acceptance criteria satisfied*
 - *Automated and manual tests pass*
 - *Code reviewed*
 - *Security, accessibility, performance, and UI criteria satisfied where applicable*
 - *Documentation and architecture views updated where necessary*
 - *Deployable and integrated into the Increment*
- An item that does **not** meet the DoD is **returned to the Product Backlog for future consideration**

Should **human review** of code, tests, etc.
be part of the DoD?

Regression Testing

- Regression testing involves re-running test cases from previous sprints to verify that existing code still functions as expected
- Why does it matter in scrum?
 - Frequent code changes may lead to *unintended side effects*
- How should it be integrated into scrum?
 - Can be part of the **DoD checklist**
 - Can be automated in continuous integration workflows via Playwright (**week 6 & 9**)



Where to Store Test Cases?

- Many workable options – pick one that suits your team and **keep it trustworthy**
 - *A team wiki (e.g. Confluence) or a spreadsheet (e.g. Google Sheets) – templates abound online*
- **Jira plug-ins** make test cases first-class work items, linked to your stories
 - *e.g. Xray or Zephyr Squad – both are paid Marketplace apps, so check before committing*
- Or keep them **next to the code** – version-controlled in your repo, reviewed in pull requests
 - *Fits nicely with automated tests (Week 6) and keeps cases from going stale*
- A test repository **nobody maintains is worse than none**, because it creates **false confidence**

Software Architecture

*from specifying system **behaviour**
to specifying system **structure***

User Story

"As a customer, I want to pay by credit card online, so that I can complete my purchase conveniently."

Acceptance Criteria

Ensure that the customer:

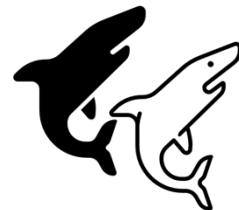
- Can input a valid card number, expiration date, CVV, and cardholder name
- Can only pay with Visa, Mastercard, or American Express
- Is requested to provide an OTP
- Has their payment processed and their order confirmed
- Is informed of payment success or failure within 2 seconds
- Has a secure TLS connection



**looks good,
let's code!**

Developer

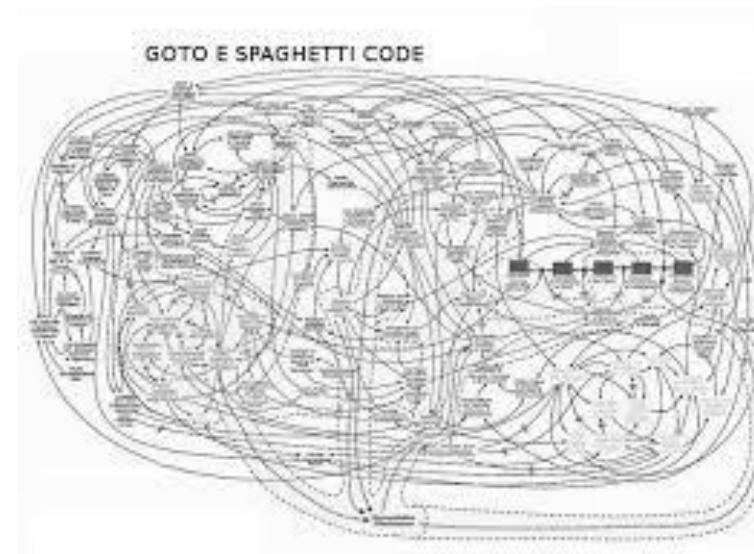
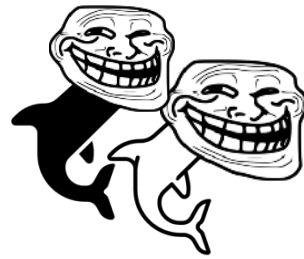
Diving into coding... without a blueprint



Diving into coding... without a blueprint



“big ball of mud”



Software Architecture

- **Software architecture** is the **high-level structure** of the system
 - *Defines the major components, their relationships, and interactions*
 - *The principles guiding their design and evolution*
- A shared **map** of the system and its important design decisions
 - *Builds a shared understanding for the team*
 - *Ensures the team (and AI) understands the context of the components they build*
- Agile projects favour code over documentation, but this **doesn't** mean that there should be no documentation
 - *We need “**just enough**” software architecture design*
 - *i.e. architectural models that are lightweight and accessible to different stakeholders*



No Architecture in Mind



Winchester Mystery House by [Www78](#). Licensed under [Creative Commons Attribution-Share Alike 3.0 Unported](#).

Too Much Architecture in Mind



Order...the result of architecture... structurally completed in 2026, its 144th year (Gaudí's centenary)

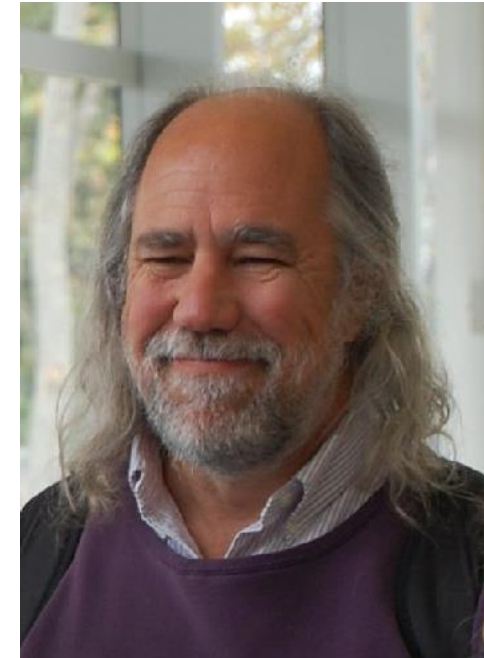
[Basilica de la Sagrada Familia](#) by [Jose Ramirez](#). Licensed under [Creative Commons Attribution-Share Alike 2.0 Generic](#).

Architecture Represents **Significant** Design Decisions

All architecture is design but not all design is architecture.

*Architecture represents the **significant design decisions** that shape a system, where significant is measured by **cost of change**.*

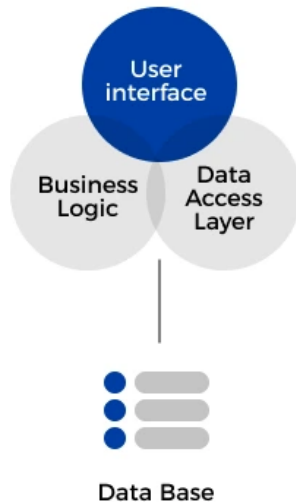
– Grady Booch, *On Design*, *Handbook of Software Architecture* blog, March 2006



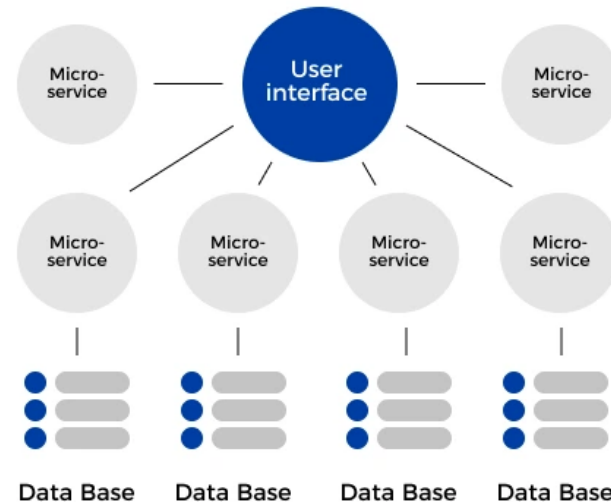
Example of Significant Design Decisions

- Recall the **software architectural patterns** you encountered in other courses – these are good examples of **significant** design choices
- For examples, **monolithic** vs. **microservices** architectures:

**MONOLITHIC
ARCHITECTURE**



**MICROSERVICE
ARCHITECTURE**



What makes this choice significant?

Agile projects favour code over documentation, but this **doesn't** mean that there should be no documentation

- We need **"just enough"** software architecture design
- i.e. architectural models that are lightweight and accessible to different stakeholders

C4 Model

*a **lean graphical modelling approach**
for software architecture*

C4 Model – *an “abstraction-first” approach*



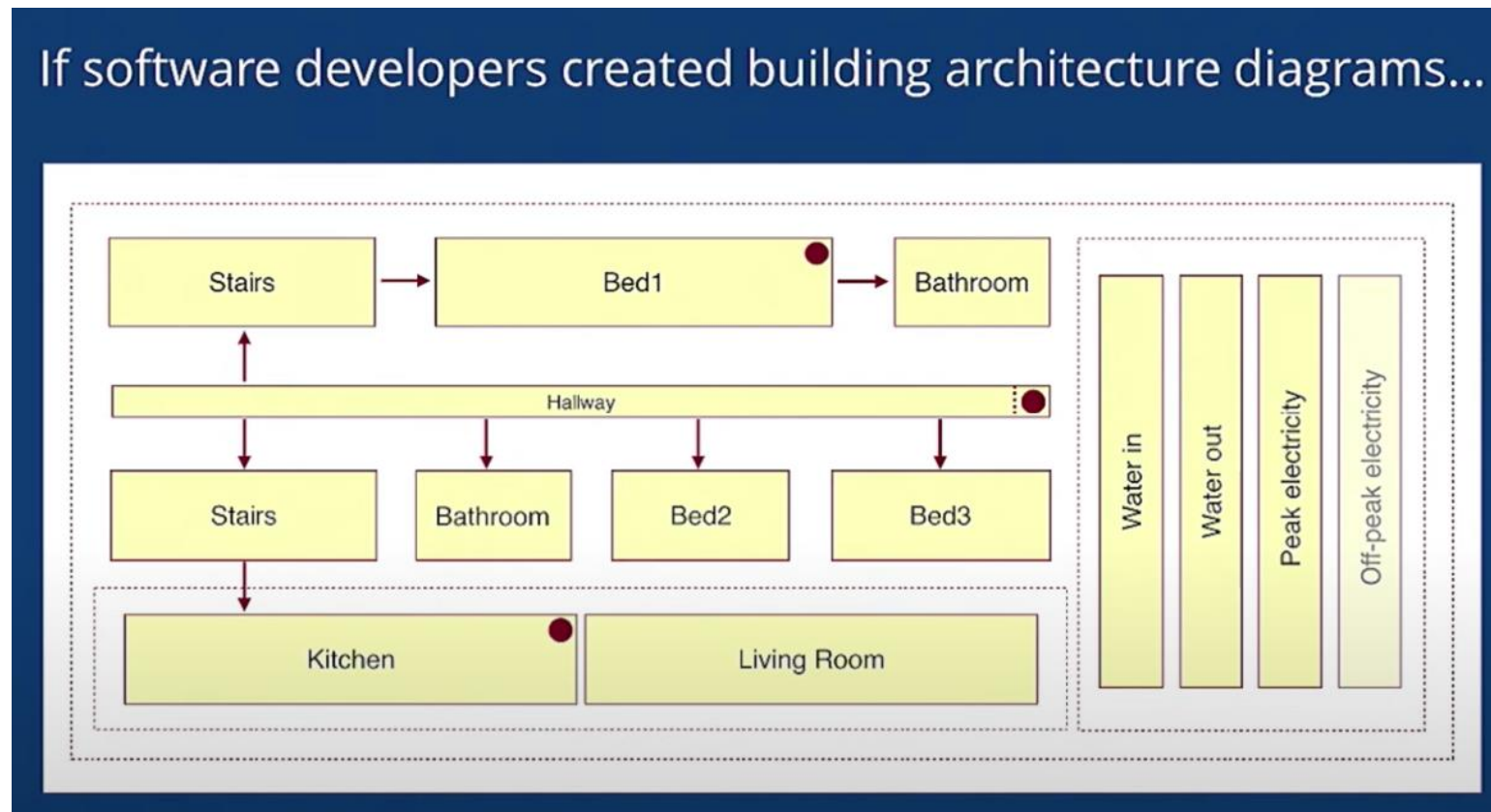
“Abstraction over notation”

what exactly
is **abstraction**?

<https://www.youtube.com/watch?v=x2-rSnhpw0g>

Aside – What is Abstraction?

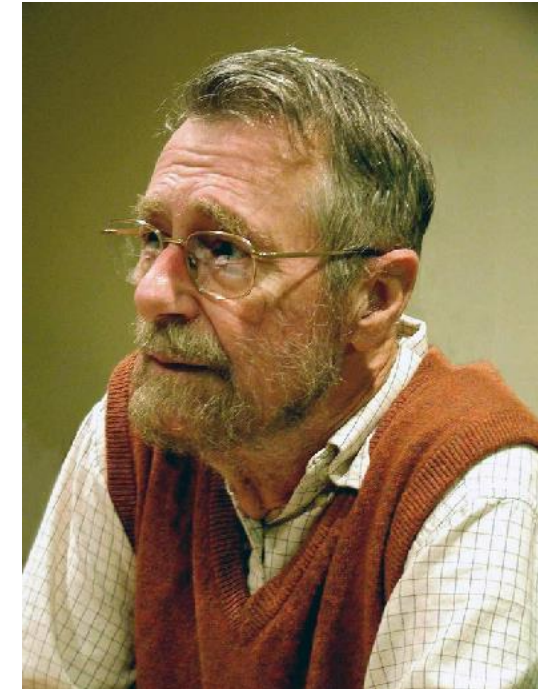
- This is abstract... but “arbitrary”
 - *The “yellow box” notation appears to mean many things...*



Aside – What is Abstraction?

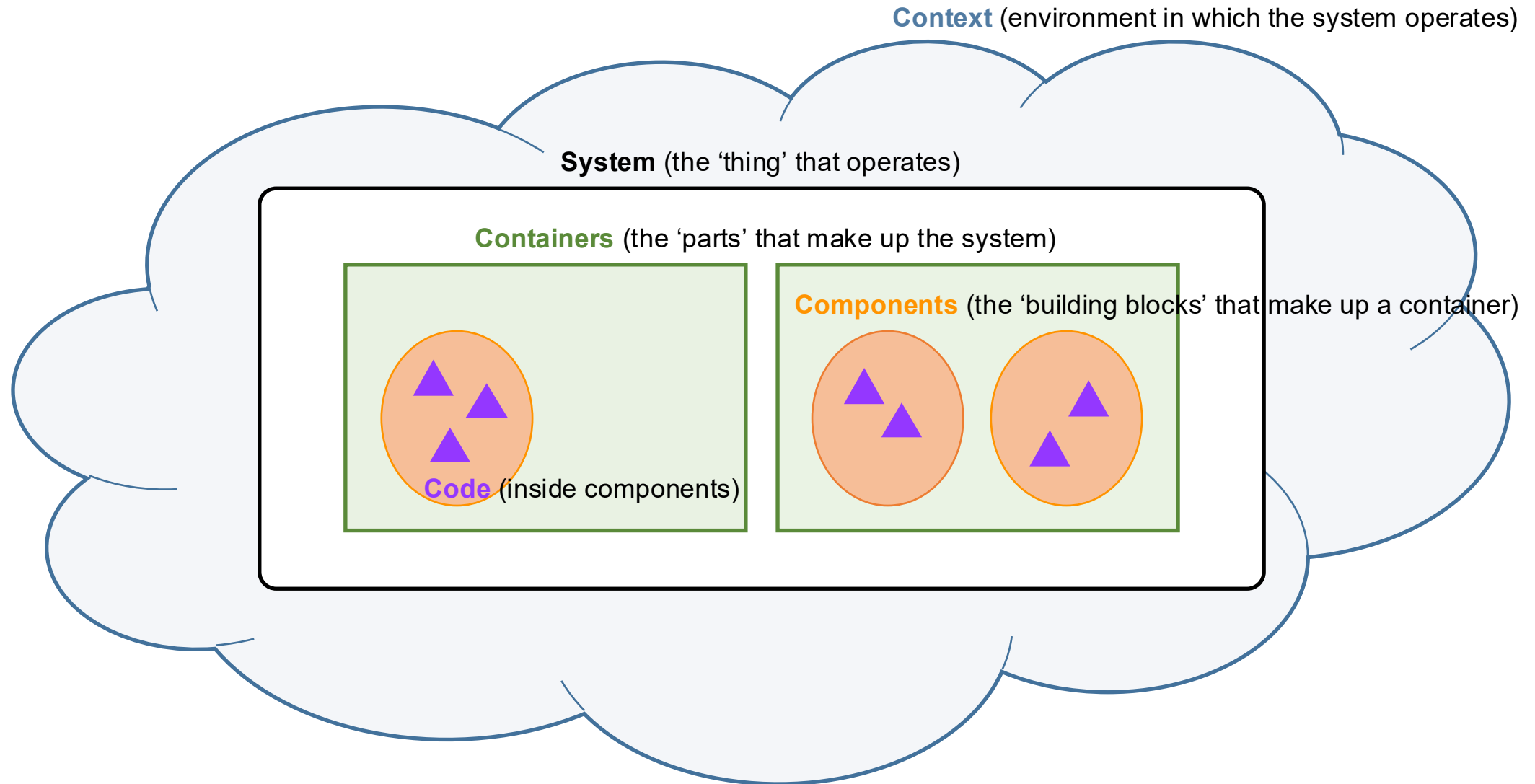
“The purpose of abstracting is **not to be vague**...
...but to create a **new semantic level** in which
one can be **absolutely precise**.”

*What do we want to be precise
about in software architecture?*



Edsger Dijkstra

C4 Model (*Context*, *Containers*, *Components*, *Code*)



C4 Model (**C**ontext, **C**ontainers, **C**omponents, **C**ode)

- The **C4 model** is a framework for visualising the architecture of a software system via a **hierarchical set of diagrams**
 - Each diagram is at a **different abstraction level**, and is intended for different stakeholders
 - Supported by draw.io, structurizr.com, [PlantUML](https://plantuml.com), and [Mermaid](https://mermaid.js.org) (experimental)
- **C1 (Context)** – high-level overview of the **environment the system operates in** (e.g. users, other systems, external services)
- **C2 (Containers)** – what are the **major containers** (applications, services, databases, ...) that make up the system
- **C3 (Component)** – **as necessary** – diagram showing the **internal structure of a container** (e.g. the structure of a single application)
- **C4 (Code)** – **as necessary** – diagram **visualising the implementation** of a component, e.g. the classes and methods of object-oriented code

Running Example – Integrated Billing for Hospitals

- Suppose you are working for a hospital that has multiple completely **separate legacy systems** for different patient costs, e.g.
 - A legacy pharmacy billing system
 - A legacy ward billing system
- Your team is developing a **modern web application** that allows all of these costs to be **retrieved and consolidated in a single place**
- What might the software architecture look like in **C4**?



Source: <https://www.healthcareittoday.com/2018/05/04/medical-billing-cartoons-fun-friday/>

C1 (Context) Diagrams

- Your system is a **single box**
 - “Zoom out” and see the big picture
 - Focus is on the **users** and (other) **systems** that it interacts with
- High level of abstraction; accessible for **non-technical** team members

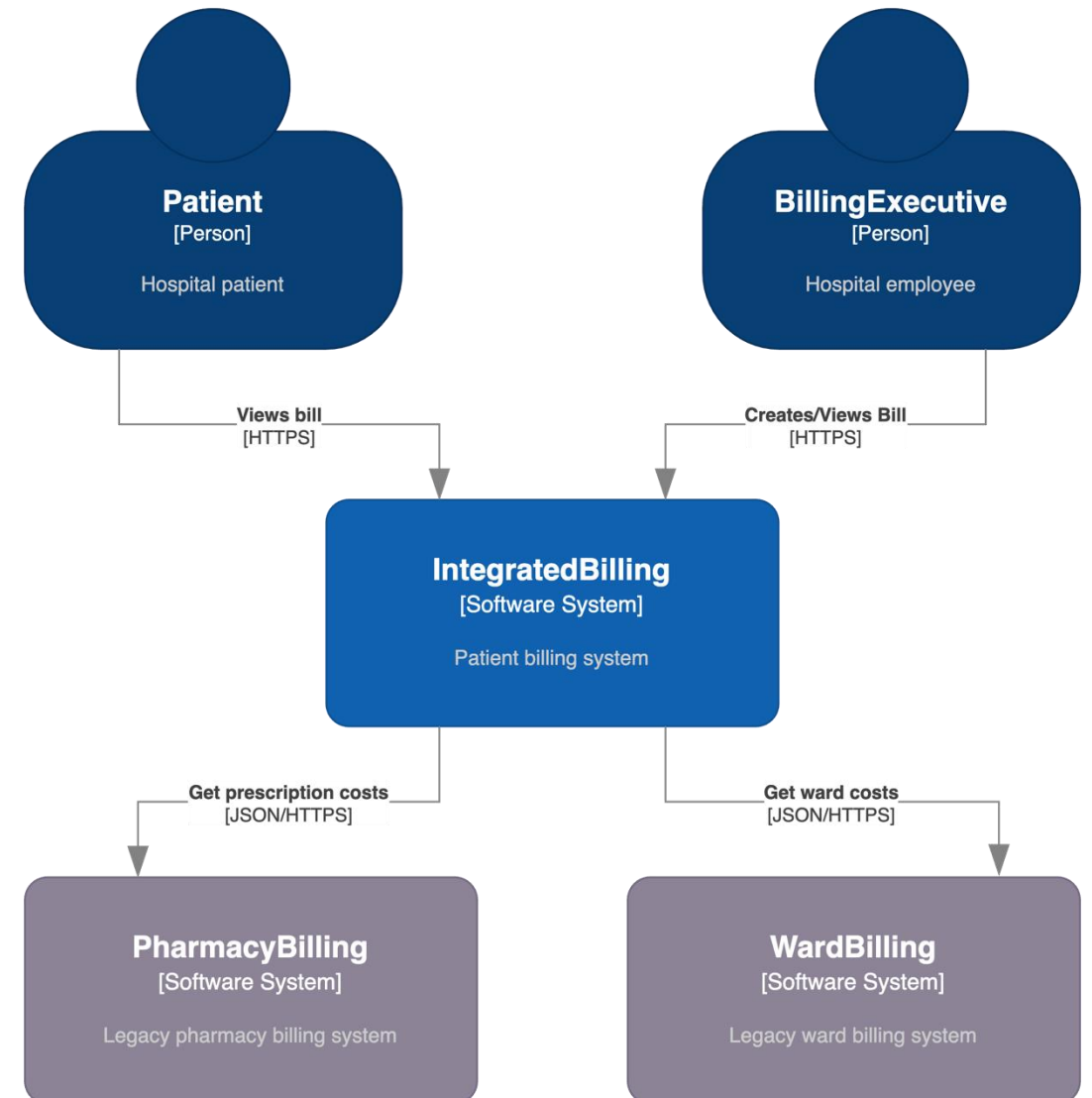
Scope: A single software system

Primary elements: The software system in scope

Supporting elements: Users (actors, roles or personas) and systems (external dependencies) that are directly connected to the software system in scope. Typically, these other systems sit outside the scope or boundary of your own software system and you do not have responsibility or ownership of them

Intended audience: Everybody, both technical and non-technical, inside and outside of the software development team

Recommended for most teams: Yes

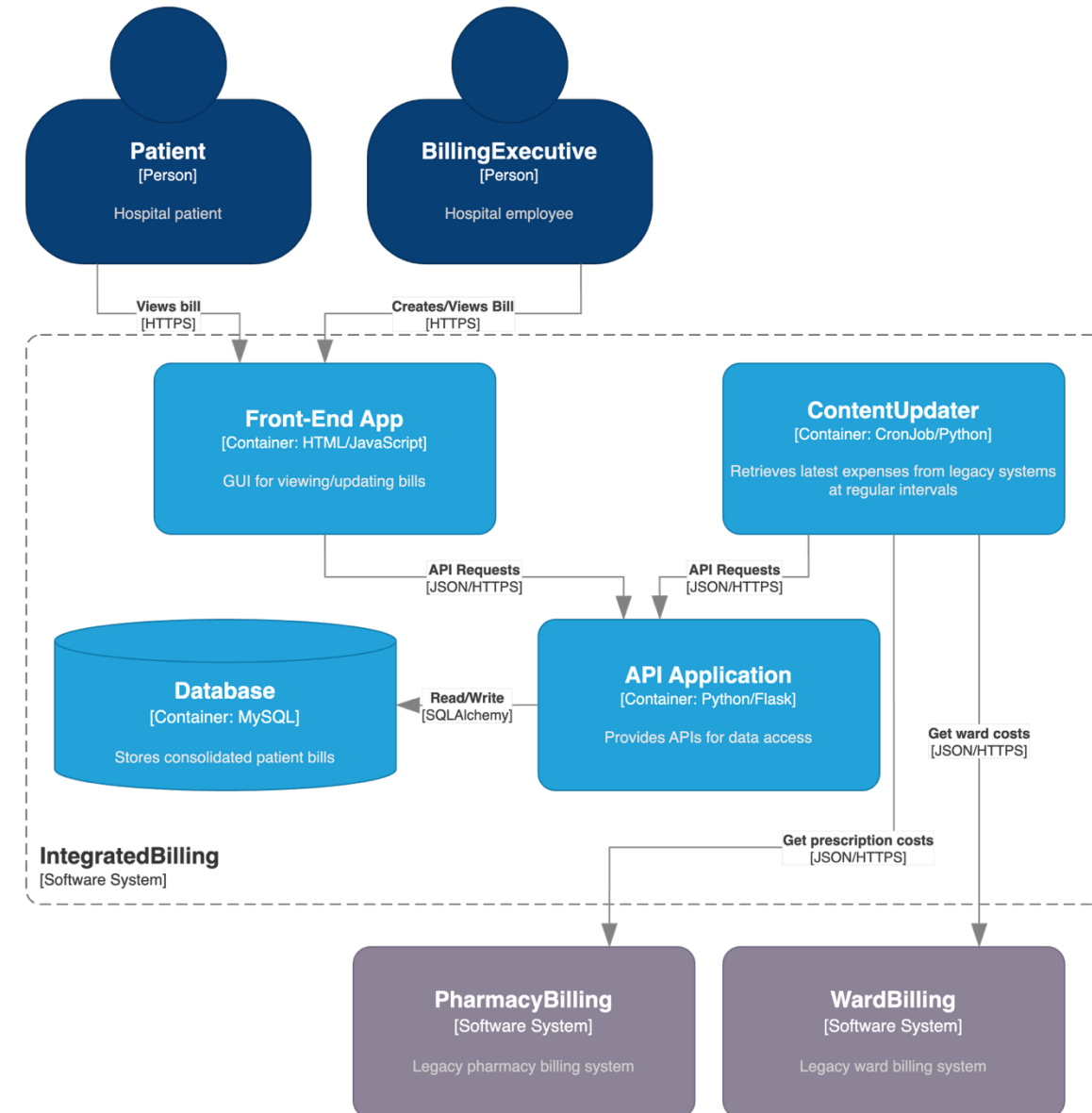


C2 (Container) Diagrams

- “Zoom in” to your **system** (the dotted line) and depict the **containers it's made up of**
- Containers are (potentially) separately running processes that execute code or store data (e.g. server-side app, database, message broker, mobile app)
- C2 depicts the “**high level shape**” of the system, and major technology choices

Scope: A single software system
Primary elements: Containers within the software system in scope
Supporting elements: People and software systems directly connected to the containers
Intended audience: Technical people inside and outside of the software development team, including software architects, developers, operations and support staff
Recommended for most teams: Yes

⚠ A C4 container is an application or data store – not necessarily a Docker container



C3 (Component) Diagrams

- “Zoom in” to a **container** (the dotted line) and depict its **internal structure**
- The components are typically **packaged together** (e.g. a single Python/Flask application)
- C3 is included **when necessary** – only needed if it adds value (e.g. visualising the structure of a particularly complex container/app)

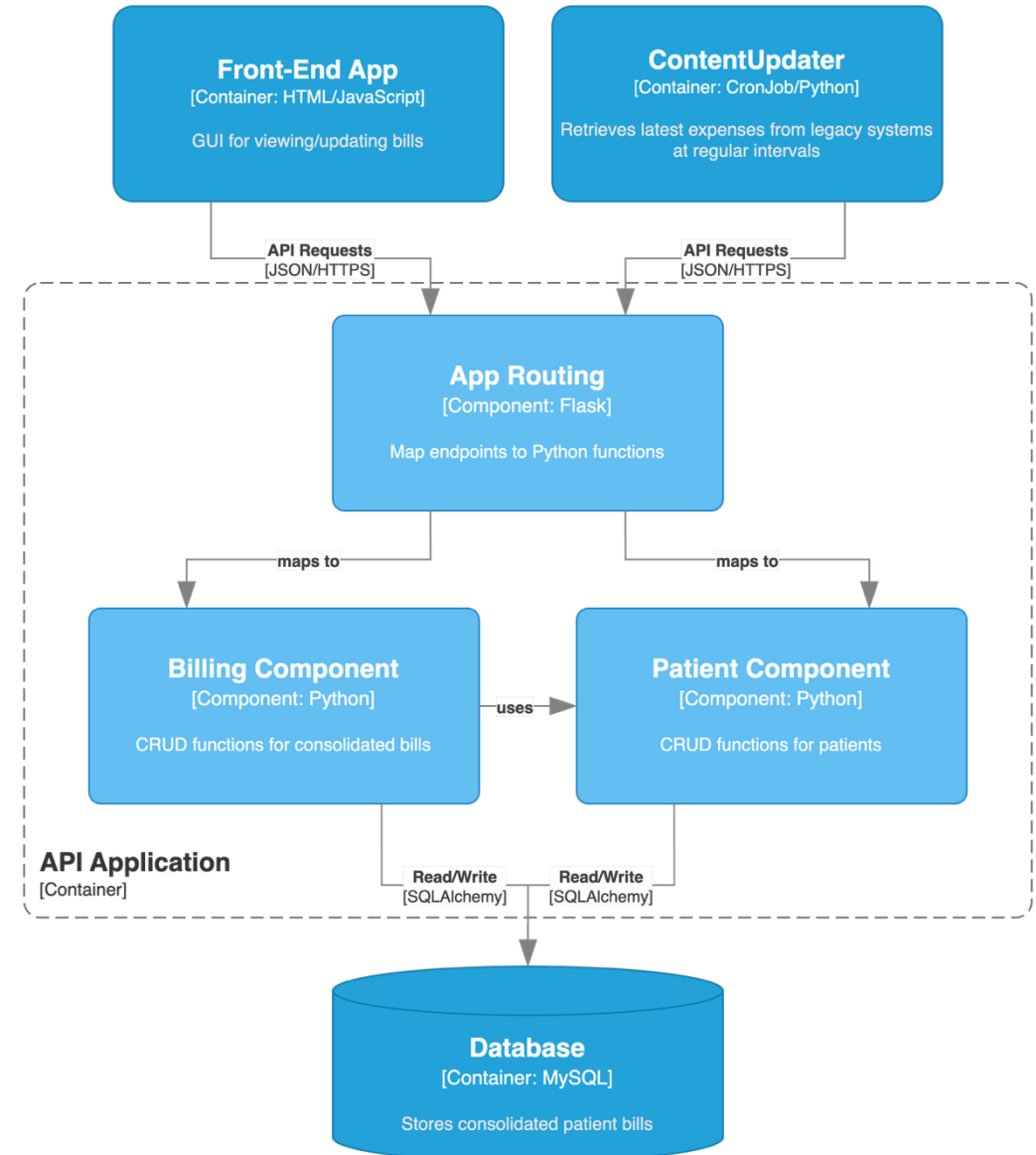
Scope: A single container

Primary elements: Components within the container in scope

Supporting elements: Containers (within the software system in scope), people and software systems directly connected to the components

Intended audience: Software architects and developers

Recommended for most teams: No. Only create component diagrams if you feel they add value



C4 (Code) Diagrams

- We can “zoom in” even further to visualise the structure of the code making up a specific component
- At this level, we can utilise existing notations, e.g.
 - *Unified Modelling Language (UML)* for modelling code
 - *Entity-Relationship (ER) Diagrams* for modelling relational databases
- Agile teams create such models more sparingly, e.g. focusing on particularly complex code structures or dependencies
- ***We will explore some of these*** – e.g. some UML in Week 5

AI Can Draw It – But Can't Decide It

- AI can generate **C4 diagrams** from a codebase or a plain-English description
- For C1 / C2, repository access alone is rarely enough to generate the complete picture – **why?**
- Ask for a **text DSL** (e.g. Structurizr DSL, C4-PlantUML), **not** a flat image
 - *Diffable in pull requests, editable, and regenerated as the design changes*

AI can propose a design, but making the high-cost-of-change decisions is **your responsibility**

```
workspace {

  !identifiers hierarchical

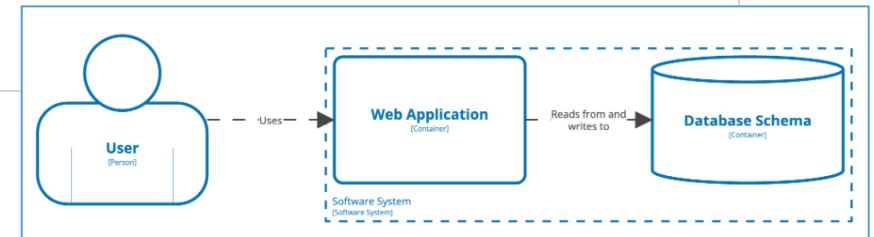
  model {
    u = person "User"
    ss = softwareSystem "Software System" {
      wa = container "Web Application"
      db = container "Database Schema" {
        tags "Database"
      }
    }

    u -> ss "Uses"
    u -> ss.wa "Uses"
    ss.wa -> ss.db "Reads from and writes to"
  }

  views {
    systemContext ss "Diagram1" {
      include *
    }

    container ss "Diagram2" {
      include *
    }

    // styling...
  }
}
```



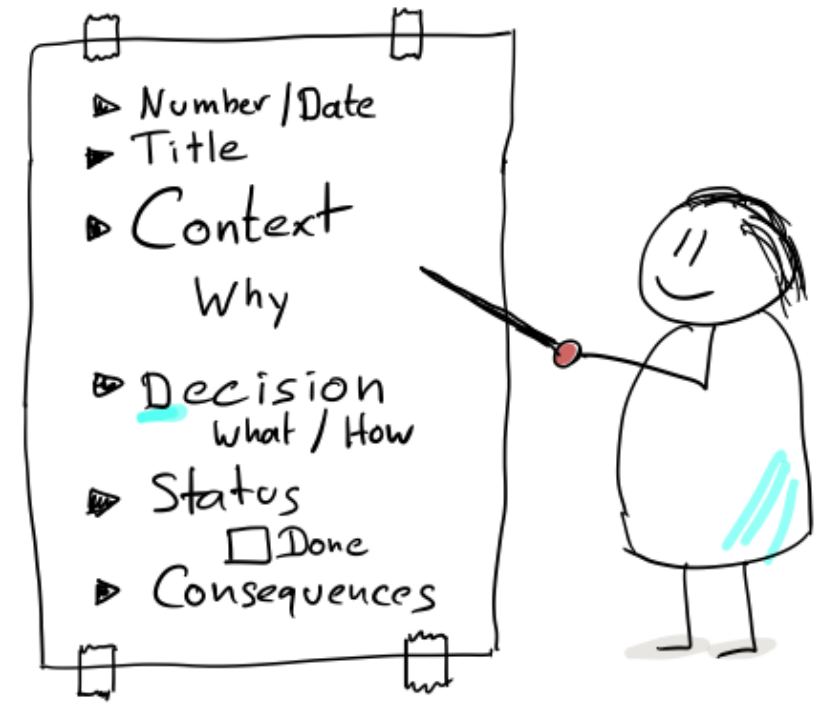
Architecture as Context for AI

- Agentic coding tools work far better against a **legible structure** than a “big ball of mud”
- Your C1 / C2 diagrams and a short architecture note become the **map an AI agent navigates by**
- Clear structure is now **doubly valuable** – humans and agents both get lost in spaghetti
- “Just enough architecture” → “just enough architecture, legible to humans **and agents**”

If an AI Agent joined your team tomorrow,
could it find its way around your system?

Architecture is Decisions, Not Only Diagrams

- A C4 diagram shows the resulting **structure** – but not **why** it is that way
- An **Architecture Decision Record (ADR)** captures one significant decision and its rationale
 - *Typical fields: context · decision · alternatives considered · consequences*
- Keep them **short and version-controlled with the code** – one file per decision
- AI can draft the ADR; the team still **owns the trade-off**



<https://www.redhat.com/en/blog/architecture-decision-records>

Lab & Reflections

Week 4 Lab

- This week's lab will guide you through **generating and reviewing C1/C2 diagrams** using AI
- In particular, we will utilise Simon Brown's **Structurizr** DSL for specifying these models
- Formative exercises (no submission)

School of Computing and Information Systems

IS212 Software Project Management



Week 4 Lab – Generating C4 Diagrams

In this lab, we will use an AI assistant to **generate** C1 (context) and C2 (container) models for three unfamiliar systems — and, more importantly, to **verify** them. Producing a diagram now takes seconds. Deciding whether it is the *right* diagram is the part that still belongs to you.

We will work in the **Structurizr DSL**, the reference implementation of the C4 model, so that a single model produces both the C1 and the C2 view. This matters: it is the same idea at two levels of abstraction, not two unrelated drawings.

Reminder – *Reflections*

- Please start filling in your weekly reflections now
- Complete them by two days after this class so that we have time to read them before we next meet 🖥️
- <http://smu.sg/is212-reflections-2026>

